BM40A1500 DATA STRUCTURES AND ALGORITHMS

RECAP AND TIPS FOR EXAM

2024

ALGORITHM ANALYSIS

- How efficient (fast) the algorithm is with respect to the size of input.
- In a typical situation, it is not important (or feasible) to define the exact number of operations (e.g., $T(n) = 3n^3+2n+10$) but it is more important to understand the type of growth asymptotic analysis and Theta notation ($\Theta(n^3)$).
- Polynomial time algorithms: $O(n^k)$
- Exponential time algorithms: $\Omega(c^n)$

- NP-completeness and NP-hard problems
  - NP-hard optimization problems are very common.
  - No efficient (polynomial time) algorithm is known → approximation algorithms

DATA STRUCTURES

- Array list, linked list, queue, stack, hash table, binary search tree, balanced search trees, heap.
- Important to select the correct data structure for the problem:
  - What operations are needed?
  - For example:
    - Fast exact-match queries → hash table
    - Efficient range queries → balanced search tree (e.g., AVL-tree)
    - Finding the task/job with the highest priority → max heap

DATA STRUCTURES

- Graphs
  - Not a very commonly-used data structure as such.
  - But very useful tool in designing algorithms:
    - Many problems can be formulated as graphs and solved using existing graph algorithms.
  - Shortest paths
    - Dijkstra's algorithm
    - Floyd's algorithm
  - Minimal cost spanning tree
    - Prim's algorithm
    - Kruskal's algorithm

ALGORITHM DESIGN PRINCIPLES

- Greedy approach
  - Locally optimal choices
  - Typically enables fast algorithm, but depending on the problem might not lead to optimal solution
- Backtracking
  - Systematic way to go trough all the possible solutions (combinations, permutations, subsets).
  - Branch-and-bound: a way to avoid testing solutions that cannot be optimal.
- Divide and conquer
  - Dividing problem into smaller subproblems that are easier to solve.
- Dynamic programming
  - Storing the solutions for the subproblems → each subproblem need to be solved only once.
- Probabilistic algorithms
  - Monte Carlo and Las Vegas algorithms

EXAM

- Paper exam
  - No programming tasks
- For the first accepted grade min. 50% of total 40 points are needed (min. 20 points are needed).
  - Grade 0: less than 20 points.
  - Grade 1: 20-23 points.
  - Grade 2: 24-27 points.
  - Grade 3: 28-31 points.
  - Grade 4: 32-35 points.
  - Grade 5: 36-40 points.
- You answer in English or in Finnish.
  - If you plan to use Finnish, I recommend to prepare for the exam by reading “Tietorakenteet ja algoritmit” by Antti Laaksonen.

EXAM

- What kind of exam questions you can expect?
  - Explaining and analyzing given algorithms (pseudocode or Python).
  - Explaining of terms (e.g., heap, dynamic programming, NP-completeness).
  - Designing an algorithm for a given problem (pseudocode is ok, but you can use Python if you want).
  - Comparing different data structures.
  - Applying a certain graph algorithm for a given graph.

- Remember to explain your answers carefully in details!

EXAM

- The first exam task will be a compulsory part of the exam. **You need to get at least 5 points out of the maximum 10 points so that the rest of the exam will be checked.**
  - The task will be about algorithm analysis:
    - What the algorithm does and how?
    - What is the asymptotic complexity of the algorithm?
    - What design principle it uses?
    - Comparing two algorithms, etc.
  - You need to show that you understand the functionality of the given codes. Therefore, you must explain your reasoning clearly.
  - You will be awarded points based on the quality of your reasoning, not the correctness of your answer.
    - For instance, merely providing the asymptotic running time without an explanation will not earn any points.

EXAM: EXAMPLE

```
function alg1(V)
N = length(vector)
D_min = inf
for i = 1 to M
    W = randomize_order(V)
    S1 = S2 = []
    for j = 1 to N
        if sum(S1) <= sum(S2)
            S1.append(W[j])
        else
            S2.append(W[j])
    D = abs(sum(S1) - sum(S2))
    if D < D_min
        D_min = D
        S1_best = S1
        S2_best = S2
return S1_best and S2_best
```

M

N

$\Theta(N)$

$\Theta(N)$

$\Theta(N^2)$

$\Theta(MN^2)$

EXAM

- **Example question 1:** what is the asymptotic running time of `alg1`?

- Good answer: "The algorithm has 2 nested for loops. The outer loop runs M times, and the inner loop runs N times. Inside the inner loop, there are sum operations that are Θ(N). The function `randomize_order` is Θ(N), but it is outside of the inner loop and does not affect on the asymptotic running time of the algorithm. Therefore, the complexity of `alg1` is $\Theta(MN^2)$."

- OK answer: "The algorithm has 2 nested for loops and a sum operation inside the loops. Therefore, the complexity of `alg1` is $\Theta(N^3)$."
  - There is a small mistake in the explanation, leading to a minor deduction in points.

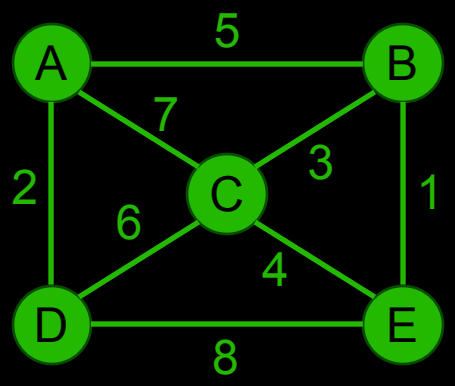
- Bad answer: $\Theta(MN^2)$
  - No explanation → 0 points.

EXAM

- **Example question 2:** what kind of algorithm design principle `alg1` uses?
- There can be multiple correct answers:
- Good answer: “The algorithm always place the next integer to the subset that has smaller sum at that point. It makes locally optimal choices at each step. Therefore, it uses greedy approach.”
- Good answer: “The algorithm uses randomization to find a better solution. The subsets are created times with randomly ordered list and the best solution is selected. The algorithm does not guarantee optimal solution, but by increasing the number of iterations the probability of good and optimal solution can be increased. Therefore, the algorithm can be seen as a Monte Carlo algorithm.”
- Bad answer: Greedy approach
  - No explanation → 0 points.

EXAM

- **Example question 3:** Construct the minimal-cost spanning tree (MCST) for the graph below using Kruskal's Algorithm. Show your step-by-step solution.



edges are sorted by weight

1)

2)

3)

4)

C-E forms a loop → do not add

5)


LUT
University